A Capability-Based Isolation Model for USB Device Access on MMU-less RTOS Using a Local WebAssembly Runtime

Xuzhen LI[1,a)] Jingjing RAO[2,b)] Tetsutaro UEHARA[2,c)]

Abstract: Real-time operating system (RTOS) platforms are widely deployed across embedded devices, where USB peripherals are commonly used for device access and data acquisition. The peripheral firmware and its vendor-specific protocol extensions are often unauditable. Even so, the host must implement USB class protocol parsers to enumerate these devices and communicate with them. These parsers process untrusted input from peripherals directly and form a large attack surface. Many such platforms have no memory management unit (MMU). Parser code then shares a flat address space with the RTOS kernel and with application state. A single out-of-bounds write can compromise the integrity of the whole system.
This paper presents a USB driver isolation model for MMU-less RTOS based on WebAssembly (WASM). Instead of virtualizing the entire driver, we isolate only the protocol parsing layer, the part that processes untrusted input directly, and run it inside a WASM instance. The host runtime keeps device state and exposes USB controller services through WASI-USB, the USB proposal of the WebAssembly System Interface (WASI). At instantiation the runtime binds capabilities only to the interfaces the module declares as imports. USB access rights thus become instance-scoped capabilities instead of shared resources. WASM itself provides instance-private linear memory with mandatory bounds checking, confining memory faults to individual instances. Consequently, both capability and memory isolation become runtime-enforced security properties without relying on MMUs, processes, or kernel isolation. The model assumes that the runtime remains the sole access path to the USB controller. At the prototype stage we also restrict the discussion to single-address-space MMU-less RTOS deployments, which keeps the kernel-level reasoning simple.
Current RTOS-oriented WASM runtimes do not support the Component Model, so an interface written in the WebAssembly Interface Type (WIT) language cannot be instantiated directly. The implementation therefore adopts Core WebAssembly with an explicit C application binary interface (ABI) shim. WIT is used solely as the interface specification, while the corresponding ABI is maintained by hand to preserve capability-oriented interface semantics. This approach retains the proposed security model while remaining compatible with resource-constrained microcontroller (MCU) platforms.

Keywords: WebAssembly, USB, capability-based isolation, MMU-less RTOS, WASI

1. Introduction

The Universal Serial Bus (USB) is an open and ubiquitous interface for connecting peripherals. Consider a typical edge AI vision system. It captures video streams from a third-party USB camera, runs person detection or face recognition locally, and uploads only feature vectors or recognition results. The camera is a commodity part, chosen for cost and availability rather than for security.

Such systems face a structural security problem. USB peripherals, like most external devices, ship with proprietary firmware, vendor-specific protocol extensions, and third-party components, and each class of device needs its own host-side driver. Auditing all of this is rarely practical. Even so, the host must implement the matching USB class protocol stack to enumerate the device and acquire data. In the example above, for a camera this means the USB Video Class (UVC), including vendor extension descriptors. The parsing layer consumes untrusted input from the peripheral directly. This makes it a primary attack surface. The problem is worse on a microcontroller (MCU) running a real-time operating system (RTOS) such as FreeRTOS. Without a memory management unit (MMU), parser code shares one flat address space with task control blocks, model parameters, and system state. A single out-of-bounds write in the parser can corrupt any part of the system.

We observe that isolating the entire driver is unnecessary. We isolate only the protocol parsing layer, the component that directly processes untrusted USB input, and place it inside a least-privilege WebAssembly instance. The host runtime retains ownership of the USB host controller, device

[1] Graduate School of Information Science and Engineering, Ritsumeikan University
[2] College of Information Science and Engineering, Ritsumeikan University
[a)] lixuzhen@cysec.cs.ritsumei.ac.jp
[b)] j-rao@fc.ritsumei.ac.jp
[c)] t-uehara@fc.ritsumei.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

management, and endpoint scheduling. It exposes these services as explicit capabilities through WASI-USB, the USB proposal of the WebAssembly System Interface (WASI). A parser holds no capabilities beyond the imports the host resolves at instantiation. As a result, the isolation model is independent of the underlying RTOS and does not require hardware memory protection or operating-system permission mechanisms.

1.1 Why an MMU-less RTOS?

We target MMU-less RTOS platforms because they offer the smallest setting in which the proposed isolation model can be evaluated. Unlike a general-purpose operating system, such a platform runs a single application, with a single USB host controller and a single owner responsible for device management. This removes process management, kernel module lifecycles, and operating-system permission policies from the design space, allowing the isolation boundary to be evaluated independently of operating-system mechanisms.

An equivalent design on Linux would have to address several further concerns. Among them are the split between kernel-space and user-space drivers, module loading and unloading, multi-process ownership of USB devices, and the interaction with mechanisms such as eBPF. These are valuable engineering problems, but they are orthogonal to the capability model proposed here.

1.2 Contributions

This paper makes two contributions.

(1) **Isolation model.** A system-independent capability-based isolation model for USB protocol parsing on MMU-less RTOS platforms, where access to a device is granted as an explicitly declared WebAssembly capability instead of a permission managed by an operating system.

(2) **Feasibility prototype.** A prototype implemented on an ESP32-S3 with ESP-IDF and the WebAssembly Micro Runtime (WAMR). It carries end-to-end USB human interface device (HID) keyboard input through an 8.6 KB WebAssembly module, showing that the isolation boundary fits within the resource budget of a commodity MCU.

2. Background and Threat Model

2.1 Background

Communication between a USB host and a device proceeds in two stages.

- **Enumeration.** When a device is attached, the host reads its device, configuration, interface, and endpoint descriptors, selects a configuration, and binds the appropriate driver.
- **Class-specific communication.** A class protocol, such as HID or UVC, defines additional class-specific descriptors, control requests, and the format of endpoint data exchanged between the host and the device.

Both stages require the host to parse data supplied by the device. In each of them the host must parse nested, variable-length structures whose length fields it cannot trust. This protocol parsing layer is the isolation target of this paper. The two stages are not treated alike in the prototype. Enumeration is performed by the native USB host stack, which parses the standard descriptors once to configure the device. The threat model below trusts this stack as part of the native base (A2). The isolated module receives the raw descriptor bytes afterwards and performs all parsing that the application consumes: the walk over configuration and class-specific descriptors and the decoding of endpoint data.

An MMU-less RTOS such as FreeRTOS schedules tasks inside one flat physical address space. There are no processes. A memory protection unit (MPU) may exist, but its regions are few and platform-specific, so it is rarely used to isolate drivers. As a result, a memory bug in any component, including a USB parser, can overwrite kernel objects, other tasks, and application data. **Fig. 1** contrasts this flat layout with the one our model produces, based on the case described in Section 1.

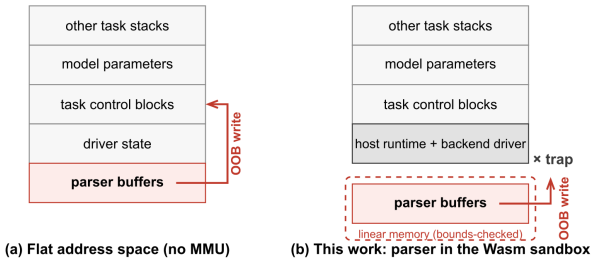


Fig. 1 Memory layout on an MMU-less RTOS: (a) flat address space; (b) this work.

According to the W3C WebAssembly Core Specification [8], WebAssembly (WASM) is a portable binary instruction format and execution model for sandboxed programs. A WASM module declares its functions, memory, and imported interfaces explicitly, allowing the runtime to validate the module before instantiation. The same module executes consistently across architectures and source languages.

Two properties are particularly important for this work. First, each WASM instance owns a private linear memory, and all memory accesses are bounds-checked by the runtime. Second, a module has no ambient authority: it cannot access files, networks, timers, or devices unless those capabilities are provided as imports by the host. Granting a capability is therefore equivalent to resolving an imported interface during instantiation.

Outside the browser, the WebAssembly Component Model [9] provides typed, language-independent interfaces through WIT, while WASI [10] standardizes common host capabilities on top of this mechanism. However, lightweight MCU runtimes have not yet adopted these standards. Our prototype uses WAMR, which exposes host functionality through its NativeSymbol mechanism and supports Core

© 2026 Information Processing Society of Japan

WASM imports only; WIT-defined Component Model interfaces are bridged through an explicit C application binary interface (ABI) shim.

Hardware access remains largely unstandardized. Although the WASI-USB proposal [11] defines WIT interfaces for USB devices, it currently provides interface definitions without embedded runtime implementations or a security model for exposing USB capabilities on MMU-less platforms. Our isolation model addresses this gap.

2.2 Threat Model

We assume a malicious or compromised USB peripheral, as in the BadUSB attack [12]. The attacker fully controls the device firmware. It can present arbitrary descriptors during enumeration and send arbitrary endpoint data afterwards. **Fig. 2** shows the two parsing stages this input reaches, descriptor parsing (S1) and report parsing (S2), together with four attack paths. Path (1) presents malformed descriptors and targets S1. Path (2) delivers malformed report data and targets S2. Path (3) is interface abuse: a parser compromised through (1) or (2) calls the host with forged handles or outside its grant. Path (4) is a replug race, in which a stale reference names a slot that a new device now occupies. In the edge AI camera above, without isolation, paths (1) and (2) end in parser code that shares its address space with the model parameters and the kernel.

Attacks at the physical layer, such as electrical overstress, are out of scope, as are side channels. The asset in scope is host integrity, which covers kernel state, other tasks, and application data such as model parameters. Availability of the USB subsystem is not defended: a compromised guest can hold its session open or block on a transfer. We leave availability out of scope as well.

We rely on four trust assumptions.

- **(A1) Runtime correctness.** The WASM runtime correctly enforces bounds checks and import resolution.
- **(A2) Trusted native base.** The native backend driver, the RTOS, and the hardware are trusted.
- **(A3) Untrusted guest.** The guest module is untrusted, and peripheral input may fully compromise it.
- **(A4) Registration order.** The capability binding layer is registered before any guest module is loaded.

Under these assumptions the model targets three properties.

- **(P1) Memory isolation.** A guest fault cannot corrupt host memory.
- **(P2) Capability isolation.** A guest can reach only the devices and endpoints it was granted.
- **(P3) Centralized arbitration.** All device access flows through one host-side entry point.

Later sections refer back to these labels.

3. Related Work

Isolating faulty or vulnerable drivers is a classic goal. Nooks [1] isolates kernel drivers with hardware protection domains, and microkernels such as MINIX 3 [2] and seL4 [3] run drivers as user-space processes. Both lines rely on an MMU. On MCUs, TockOS [4] and ACES [5] use the MPU to separate untrusted components, but MPU regions are scarce and coarse. Software fault isolation (SFI) [6] confines memory access without hardware support. WASM is a modern, portable form of SFI [7], and our model builds on it.

BadUSB [12] showed that peripheral firmware can turn a commodity device into an attacker. Later work vets or filters USB traffic by policy: GoodUSB [13] and USBFILTER [14] mediate which devices and interfaces may attach. Cinch [15] places USB protection domains behind a virtualized gateway, which requires an MMU and virtualization support. FirmUSB [16] analyzes peripheral firmware offline, and USBFuzz [17] found dozens of memory bugs in host-side USB stacks, confirming that the parsing layer is a real-world attack surface. These approaches are orthogonal to ours: they decide which devices to trust, while we contain the damage of parsing an untrusted one.

Several WASM runtimes target MCUs. WAMR [18], WARDuino [20], and wasm3 [19] all run on devices of this class, and Sledge [21] applies WASM to lightweight isolation in the cloud and at the edge. A runtime by itself, however, is not an isolation boundary, and wasm3 makes the gap concrete. It is a compact interpreter that enforces the bounds checks the WASM semantics require, so an access performed by guest bytecode cannot leave linear memory. Those checks stop at the host boundary. When a guest passes a pointer to a host function, the interpreter yields a raw address into linear memory. Checking that address against the instance bounds is left to whoever wrote the function. An interpreter therefore supplies the sandbox but not the capability boundary. How much isolation a system actually gets depends on how the host exposes its functions. That is the layer this paper specifies rather than assumes.

Wasefire [22] and Aerogel [23] are the closest in mechanism to this work. Both run WASM on MMU-less devices, and both make the host the party that grants access to hardware. They differ from us in where the boundary is drawn and in which party is untrusted. Wasefire builds firmware from applets and sandboxes each applet from the others and from the platform, while the platform and the board API beneath it stay native and trusted. Aerogel targets multi-tenant devices, where each WASM application is a tenant, and defines fine-grained policies over sensors, actuators, energy, and memory. In both systems the sandboxed party is application code supplied by someone else, and the driver stack sits on the trusted side of the boundary.

We place the boundary inside the driver stack instead. The parser we isolate is host code, written by the same party as the runtime, and it is not the attacker. The attacker is the peripheral, and the parser is simply the code the attacker reaches first. We apply isolation so that a compromise of the parser cannot spread. Capabilities are scoped to the device and endpoints that parser was granted, not to a tenant or an application. Aerogel also expresses access control as a policy layer above the runtime, whereas the capability

© 2026 Information Processing Society of Japan

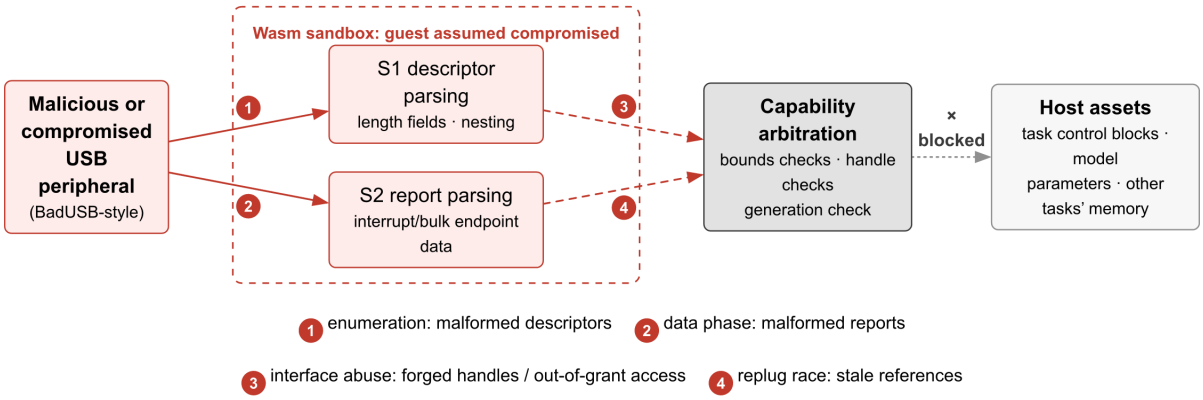


Fig. 2 Threat model: parsing stages S1 and S2, attack paths (1)–(4), and the isolation boundary.

boundary here is the import list of the module itself, so no policy language is introduced.

To our knowledge, no prior work isolates the USB protocol parsing layer of the host driver stack on an MMU-less RTOS. **Table 1** summarizes the comparison.

Table 1 Comparison with related isolation approaches.

System	Mechanism	HW	Target	Port.
Nooks	HW domain	MMU	driver	no
MINIX 3, seL4	process	MMU	driver	no
TockOS, ACES	MPU region	MPU	component	ltd.
Cinch	gateway VM	MMU + virt.	USB stack	no
Wasefire	WASM	none	applet	yes
Aerogel	WASM + policy	none	tenant app	yes
This work	WASM cap.	none	parser	yes

4. Prototype Implementation

To verify the isolation model, we implemented a proof of concept on an ESP32-S3. The prototype isolates the USB HID protocol parser inside a WebAssembly module, while the native firmware retains exclusive control of the USB host controller and device management.

Table 2 lists the prototype environment. The guest is loaded as a portable `.wasm` binary and executed by WAMR's fast interpreter, so no ahead-of-time compilation step is involved. The isolated parser is a `no_std` Rust crate, and the firmware build embeds the resulting module as a binary section. **Fig. 3** shows the resulting structure.

4.1 Layering and Startup Order

The native side is split into two layers that never mix concerns. The backend wraps the asynchronous USB host client of ESP-IDF into a set of blocking calls and owns all device state. It contains no WebAssembly concept at all. The

Table 2 Prototype environment.

Component	Version / setting
MCU	ESP32-S3
SDK	ESP-IDF 6.0.2
WASM runtime	WAMR 2.4.5
Execution mode	fast interpreter
Guest target	`wasm32-unknown-unknown`
Build tooling	Python 3.11.9
Guest module	8571 bytes

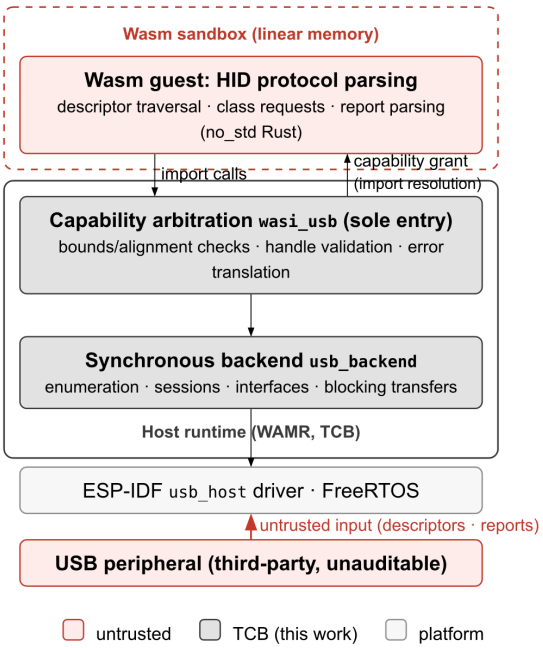


Fig. 3 System architecture and trust boundary. The arbitration layer is the sole entry to device access.

binding layer above it does only four things: it exposes one native function per import name, checks pointer arguments, encodes and decodes handles, and translates `esp_err_t` into the error codes of the interface. The isolated parser sits above both and reaches them only through imports. Every device access thus flows through this single native entry path, which realizes property (P3).

Startup order enforces assumption (A4). The firmware entry point initializes the runtime, registers the `wasi_usb` and `env` native modules, and only then loads the guest module. WAMR requires this order. It also means that an import the host never registered cannot be resolved, so a module asking for a capability it was not granted fails at instantiation, not at first use. The guest itself is loaded in a dedicated FreeRTOS task, so a failure to load or a trap during execution takes down that task alone.

© 2026 Information Processing Society of Japan

4.2 Capability Binding and Handles

Each import is bound through the WAMR NativeSymbol mechanism, which pairs a wire name with a native function pointer and a signature string. Resolution happens once, at instantiation, so the set of capabilities an instance holds is fixed and cannot grow afterwards. This is the enforcement behind property (P2), although the granted set is currently the whole interface, as noted below.

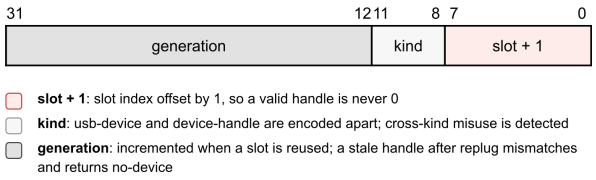


Fig. 4 Encoding of a 32-bit device handle: slot index plus one (bits 7–0), object kind (bits 11–8), and generation counter (bits 31–12).

Device references crossing the boundary are not table indices but integers that pack three fields, laid out in **Fig. 4**. The backend keeps a fixed array of device slots, and a handle names one of them. Bits 7–0 hold that slot index plus one, so slot zero is still addressable while the value zero never denotes a valid handle. Bits 11–8 hold an object kind. It separates a device reference from an open session handle, so passing one where the other is expected is a detectable error instead of a type confusion. Bits 31–12 hold a generation counter that the backend increments whenever a slot is reused. The generation keeps the boundary safe across unplug events. A handle retained across a disconnect fails to match the current generation, so the call returns `no-device` instead of quietly operating on whichever device now occupies the slot. A handle is nothing but an encoding of a slot and a generation, so the host keeps no separate handle table. Malformed references fail in the same way: an unknown slot, a wrong kind, and a stale generation are all rejected before reaching the backend. The encoding detects misuse and stale references; it is not a secret. In the current prototype a single instance is granted the interface as a whole, and the backend admits one session per device at a time. Restricting an instance to a named subset of devices at instantiation is a natural extension of the same mechanism, but it is not implemented here.

4.3 Boundary Checks

The signature string tells WAMR how to check each pointer. A buffer written as `*~` is checked together with the length that follows it. A bare `*` guarantees only one readable byte. Every argument dereferenced as a structure is re-checked in the binding layer against the instance bounds. Alignment is checked as well: the guest supplies the offset, and the binding layer dereferences these buffers as typed structures whose natural alignment the ABI requires. Since the base of linear memory is aligned, alignment of the native address is equivalent to alignment of the guest offset. Both failures are ABI violations, not ordinary errors, so they raise a trap instead of returning a code. The trap terminates the instance and leaves host state untouched, which is property (P1) in operational form.

4.4 The Isolated Parser

All HID logic lives in the guest. It calls `init`, polls `list-devices`, and for each device requests the active configuration descriptor. The host returns the raw descriptor bytes instead of a parsed structure, which keeps the parsing work on the guest side of the boundary. The native stack has already walked these descriptors once during enumeration; as stated in the threat model, that parse belongs to the trusted base (A2). The guest then walks that byte array itself, stepping descriptor by descriptor through the `bLength` and `bDescriptorType` header to find the first HID interface and its interrupt IN endpoint. This walk is the code the threat model targets: every bound in it comes from the device. Having found an interface, the guest opens a session and claims it. For boot-subclass devices it issues `SET_PROTOCOL` and `SET_IDLE`. It then decodes report bytes into keystrokes. The host is not aware that any of this is HID. It sees a sequence of capability invocations and a blocking transfer on an endpoint that the guest was granted.

The consequence is that a memory error anywhere in descriptor walking or report decoding is confined to the instance. The only addresses the parser can name are offsets into its own linear memory, so kernel objects and other tasks stay out of reach. The only host functions it can call are the ten USB imports it declared, plus three helpers for logging and delay.

5. Conclusion and Future Work

We isolated the USB protocol parsing layer of a host driver stack inside a WebAssembly instance on an MMU-less RTOS. The host keeps the controller, the device state, and endpoint scheduling, and the parser receives only the capabilities it declares as imports. Memory isolation and capability isolation are enforced by the runtime, not by an MMU, a process boundary, or an operating-system permission mechanism. Because no runtime for this class of device implements the Component Model, WIT serves as the specification while a Core WASM C ABI is carried below it as an explicit shim. A prototype on ESP32-S3 carries HID keyboard input end to end through an 8.6 KB module, which shows that the boundary fits within the resource budget of a commodity MCU.

The prototype is deliberately narrow and has four main limits.

- **Capability granularity.** The interface is granted as a whole to a single instance, and the backend admits one session per device. Restricting an instance to a named subset of devices at instantiation is not implemented. Capabilities are scoped to the instance, but not yet to the device.
- **Interface coverage.** Only the subset of WASI-

© 2026 Information Processing Society of Japan

USB needed to drive one HID device end to end is mapped. The deferred functions are recorded, not silently omitted, but configuration switching, hotplug events, and concurrent in-flight transfers are among them. Enumeration-time descriptor parsing also remains in the trusted native stack; only the parsing after enumeration is isolated.

- **No quantitative evaluation.** Interpretation overhead, memory footprint, and transfer latency are not measured in this paper. The claim made here is feasibility, not performance.
- **Single platform.** The design uses no operating-system permission mechanism, so independence from the RTOS holds by construction. The prototype, however, has run on one MCU and one runtime only.

We see three directions for future work. The first is abstraction. The mapping rules are already written down, so the shim and the guest-side bindings should be generated from WIT instead of maintained by hand. Once a runtime for this class of device implements the Component Model, they can be dropped altogether. The second is to complete the host-side security logic. That means granting a named set of devices at instantiation, arbitrating between several parser instances, and defining what an instance may observe about devices it was not granted. The third is to carry more class protocols across the boundary, UVC in particular. A video parser is both larger than a boot-protocol keyboard and closer to the motivating scenario. Together these would turn the present feasibility result into a general isolation layer for untrusted peripheral input.

References

[1] Swift, M. M., Bershad, B. N. and Levy, H. M.: Improving the Reliability of Commodity Operating Systems, *Proc. SOSP '03*, pp.207–222 (2003).

[2] Herder, J. N., Bos, H., Gras, B., Homburg, P. and Tanenbaum, A. S.: MINIX 3: A Highly Reliable, Self-Repairing Operating System, *ACM SIGOPS Operating Systems Review*, Vol.40, No.3, pp.80–89 (2006).

[3] Klein, G., Elphinstone, K., Heiser, G., et al.: seL4: Formal Verification of an OS Kernel, *Proc. SOSP '09*, pp.207–220 (2009).

[4] Levy, A., Campbell, B., Ghena, B., Giffin, D. B., Pannuto, P., Dutta, P. and Levis, P.: Multiprogramming a 64kB Computer Safely and Efficiently, *Proc. SOSP '17*, pp.234–251 (2017).

[5] Clements, A. A., Almakhdhub, N. S., Bagchi, S. and Payer, M.: ACES: Automatic Compartments for Embedded Systems, *Proc. USENIX Security '18*, pp.65–82 (2018).

[6] Wahbe, R., Lucco, S., Anderson, T. E. and Graham, S. L.: Efficient Software-Based Fault Isolation, *Proc. SOSP '93*, pp.203–216 (1993).

[7] Haas, A., Rossberg, A., Schuff, D. L., Titzer, B. L., Holman, M., Gohman, D., Wagner, L., Zakai, A. and Bastien, J. F.: Bringing the Web up to Speed with WebAssembly, *Proc. PLDI '17*, pp.185–200 (2017).

[8] Rossberg, A. (ed.): WebAssembly Core Specification, *W3C Recommendation* (2019), available at https://www.w3.org/TR/wasm-core-1/ (accessed 2026-08-21).

[9] WebAssembly Community Group: WebAssembly Component Model, available at https://github.com/WebAssembly/component-model (accessed 2026-08-21).

[10] WebAssembly Community Group: WASI: The WebAssembly System Interface, available at https://github.com/WebAssembly/WASI (accessed 2026-08-21).

[11] Sebrechts, M. et al.: WASI USB: A Proposed WebAssembly System Interface API, available at https://github.com/WebAssembly/wasi-usb (accessed 2026-08-21).

[12] Nohl, K. and Lell, J.: BadUSB: On Accessories that Turn Evil, *Black Hat USA* (2014).

[13] Tian, D. J., Bates, A. and Butler, K.: Defending Against Malicious USB Firmware with GoodUSB, *Proc. ACSAC '15*, pp.261–270 (2015).

[14] Tian, D. J., Scaife, N., Bates, A., Butler, K. and Traynor, P.: Making USB Great Again with USBFILTER, *Proc. USENIX Security '16*, pp.415–430 (2016).

[15] Angel, S., Wahby, R. S., Howald, M., Leners, J. B., Spilo, M., Sun, Z., Blumberg, A. J. and Walfish, M.: Defending against Malicious Peripherals with Cinch, *Proc. USENIX Security '16*, pp.397–414 (2016).

[16] Hernandez, G., Fowze, F., Tian, D., Yavuz, T. and Butler, K. R. B.: FirmUSB: Vetting USB Device Firmware using Domain Informed Symbolic Execution, *Proc. CCS '17*, pp.2245–2262 (2017).

[17] Peng, H. and Payer, M.: USBFuzz: A Framework for Fuzzing USB Drivers by Device Emulation, *Proc. USENIX Security '20*, pp.2559–2575 (2020).

[18] Bytecode Alliance: WebAssembly Micro Runtime (WAMR), available at https://github.com/wasm-micro-runtime/wasm-micro-runtime (accessed 2026-08-21).

[19] Massey, S. and Shymanskyy, V.: Wasm3: A Fast WebAssembly Interpreter and the Most Universal WASM Runtime, available at https://github.com/wasm3/wasm3 (accessed 2026-08-21).

[20] Gurdeep Singh, R. and Scholliers, C.: WARDuino: A Dynamic WebAssembly Virtual Machine for Programming Microcontrollers, *Proc. MPLR '19*, pp.27–36 (2019).

[21] Gadepalli, P. K., McBride, S., Peach, G., Cherkasova, L. and Parmer, G.: Sledge: A Serverless-First, Light-Weight WASM Runtime for the Edge, *Proc. Middleware '20*, pp.265–279 (2020).

[22] Google: Wasefire: A Secure Firmware Platform, available at https://github.com/google/wasefire (accessed 2026-08-21).

[23] Liu, R., Garcia, L. and Srivastava, M.: Aerogel: Lightweight Access Control Framework for WebAssembly-Based Bare-Metal IoT Devices, *Proc. IEEE/ACM SEC '21*, pp.94–105 (2021).

© 2026 Information Processing Society of Japan